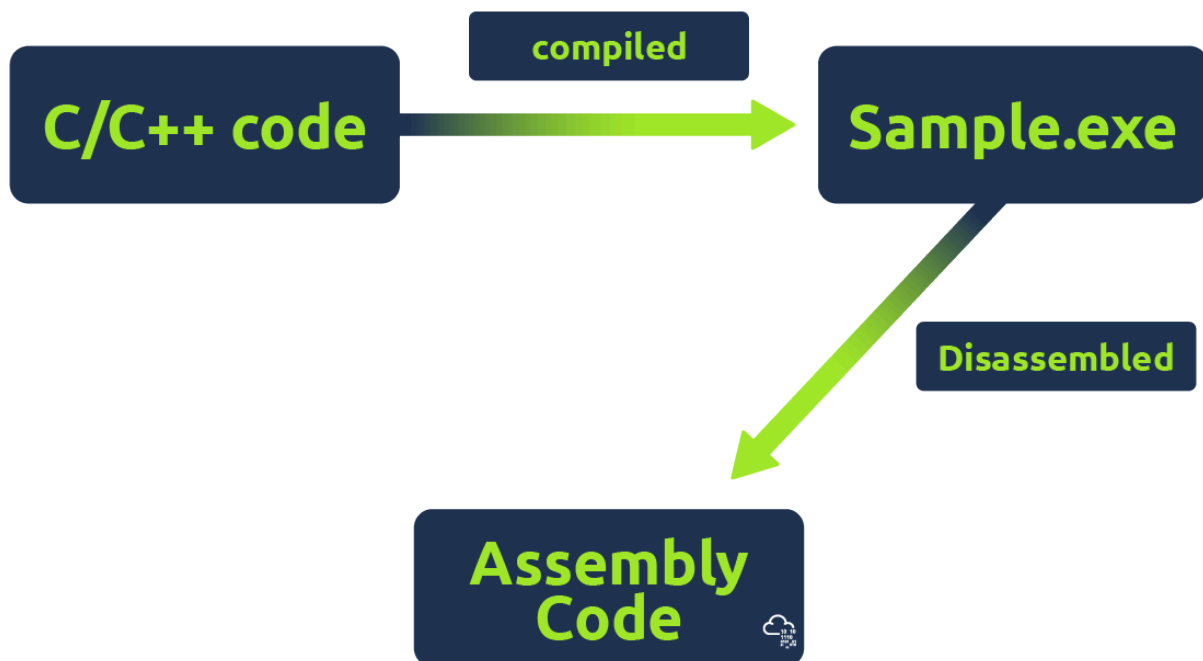


L2 - 13.3. Advanced Static Analysis

Task 1 - Introduction

Analyzing disassembled code to uncover malware's core functionality.

Advanced Static Analysis: Advanced static analysis is a technique used to analyze the code and structure of malware without executing it. This can help us identify the malware's behavior and weaknesses and develop signatures for antivirus software to detect it. By analyzing the code and structure of malware, researchers can also better understand how it works and develop new techniques for defending against it.



Task 2 - Malware Analysis: Overview

Malware analysis is the process of examining malicious software (malware) to understand how it works and identify its capabilities, behavior, and potential

impact. There are four main steps in analyzing malware: **basic static analysis**, **basic dynamic analysis**, **advanced static analysis**, and **advanced dynamic analysis**. Each step uses different tools and techniques to gather information about the malware.

Basic Static Analysis: The basic static analysis aims to understand the malware's structure and behavior without executing it. This involves examining the malware's code, file headers, and other static properties.

Basic Dynamic Analysis: The basic dynamic analysis aims to observe the malware's behavior during execution in a controlled environment. This involves executing the malware in a sandbox or lab machine and monitoring its system activity, network traffic, and process behavior.

Advanced Dynamic Analysis: The advanced dynamic analysis aims to uncover more complex and evasive malware behavior using advanced monitoring techniques. This involves using more sophisticated sandboxes and monitoring tools to capture the malware's behavior in greater detail.

Advanced Static Analysis: The advanced static analysis aims to uncover hidden or obfuscated code and functionality within the malware. This involves using more advanced techniques to analyze the malware's code, such as deobfuscation and code emulation.

How Advanced Static Analysis Is Performed

Advanced static analysis uses disassemblers (like IDA Pro, Binary Ninja, or radare2) to examine malware's code structure, entry points, system calls, and control flow graph to uncover its core capabilities, attack vectors, evasion techniques, and potential threat level.

- Identify the entry point of the malware and the system calls it makes.
- Identify the malware's code sections and analyze them using available tools such as debuggers and hex editors.
- Analyze the malware's control flow graph to identify its execution path.
- Trace the malware's dynamic behavior by analyzing the system calls it makes during execution.
- Use the above information to understand the malware's evasion techniques and the potential damage it can cause.

Answer the questions below

Does advanced static analysis require executing the malware in a controlled environment?

no

Task 3 - Connecting to the VM

Start the lab machine by clicking the **Start Lab Machine** button.

Username	Administrator
Password	Passw0rd!
IP	MACHINE_IP

Answer the questions below

How many files are present in the Code_Constructs folder on the Desktop?

5

Task 4 - Ghidra: A Quick Overview

Ghidra is a software reverse engineering tool that allows users to analyze compiled code to understand its functionality. It is designed to help analysts and developers understand how software works by providing a platform for decompiling, disassembling, and debugging binaries.

Features: Ghidra includes many features that make it a powerful reverse engineering tool. Some of these features include:

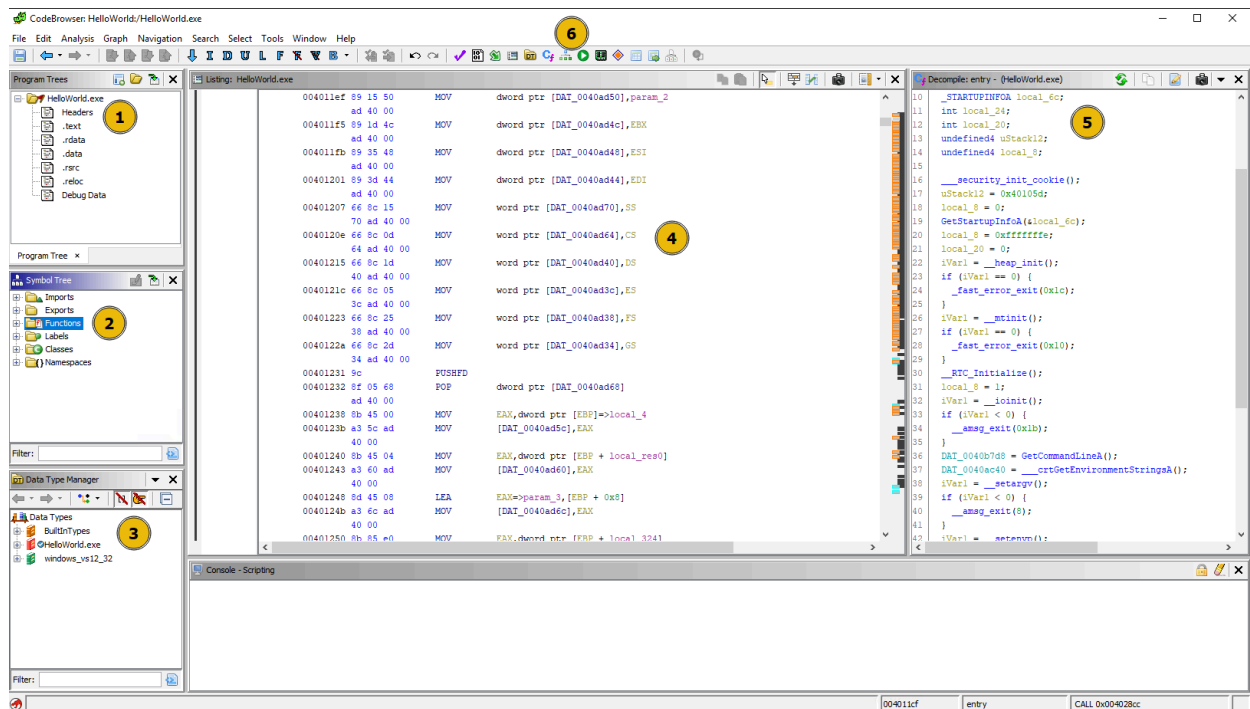
- **Decompilation:** Ghidra can decompile binaries into readable C code, making it easier for developers to understand how the software works.
- **Disassembly:** Ghidra can disassemble binaries into assembly language, allowing analysts to examine the low-level operations of the code.
- **Debugging:** Ghidra has a built-in debugger that allows users to step through code and examine its behavior.
- **Analysis:** Ghidra can automatically identify functions, variables, and other code to help users understand the structure of the code.

How to use Ghidra for Analysis

- Open Ghidra and create a new project.
- Select **Non-Shared Project**. Selecting the Shared Project would allow our analysis with other analysts.
- Name the project and set the directory, or leave the default path.
- Import the malware executable you want to analyze. Now that we have created an empty project, let's drag & drop `HelloWorld.exe` from the Desktop into that project, or navigate to the Desktop folder and select the program.
- Once it's imported, it shows us the summary of the program as shown below:
- Double-click on **HelloWorld.exe** to open it in the Code Browser. When asked to analyze the executable, click Yes.
- The next window that appears shows us various analysis options. We can check or uncheck them based on our needs. These plug-ins or add-ons assist Ghidra during the analysis.

Exploring the Ghidra Layout

- Ghidra has so many options to aid in our analysis. The default layout is shown and briefly explained below.

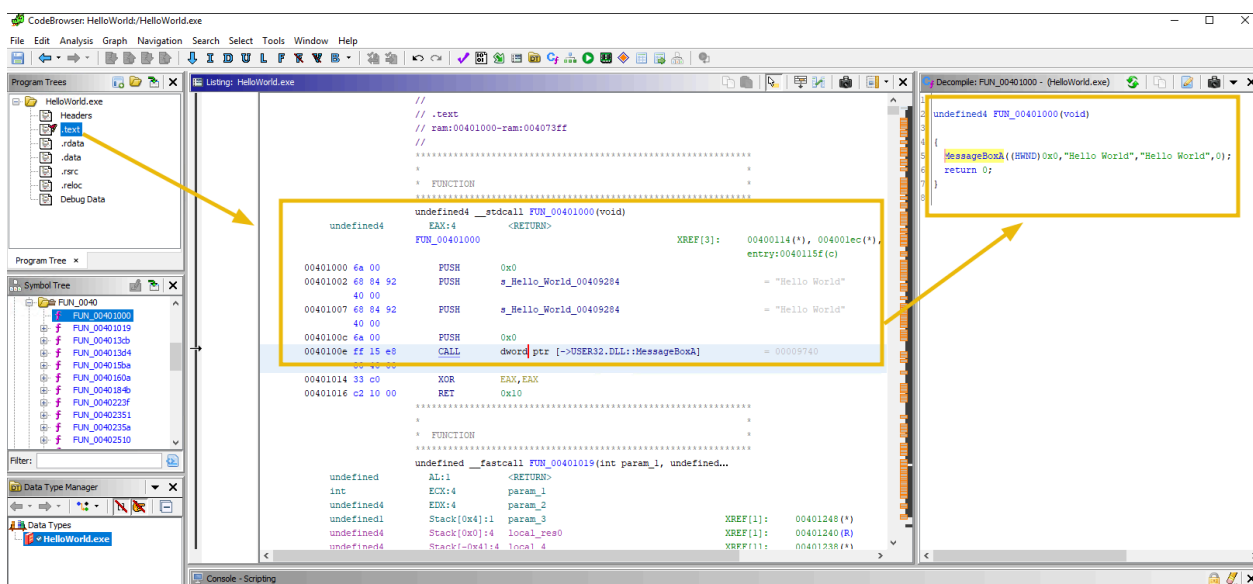


1. **Program Trees:** Shows sections of the program.
2. **Symbol Tree:** Contains important sections like Imports, Exports, and Functions.
 - **Imports:** This section contains information about the libraries being imported by the program. Clicking each API call shows the assembly code that uses it.
 - **Exports:** This section contains the API/function calls being exported by the program. This section is useful when analyzing a DLL, as it will show all the functions dll contains.
 - **Functions:** This section contains the functions it finds within the code. Clicking each function takes us to its disassembled code.
3. **Data Type Manager:** This section shows various data types found in the program.
4. **Listing:** This window shows the disassembled code of the binary, which includes the following values in order.
 - Virtual Address
 - Opcode

- Assembly Instruction (PUSH, POP, ADD, XOR, etc.)
 - Operands
 - Comments
5. **Decompile:** Ghidra translates the assembly code into pseudo-C code here. This is a very important section to examine during analysis, as it provides a better understanding of the assembly code.
6. **Toolbar:** It has various options to use during the analysis.
- **Graph View:** The Graph View in the toolbar is an important option, allowing us to see the graph view of the disassembly.
 - **The Memory Map** option shows the memory mapping of the program as shown below:
 - This navigation toolbar shows different options to navigate through the code.
 - Explore Strings. Go to **Search -> For Strings**, and click **Search** to find the strings that Ghidra finds within the program. This window can contain very juicy information to help us during the analysis.

Analyzing HelloWorld in Assembly.

The disassembled section shows how the arguments are pushed, followed by the call to MessageBoxA, which displays the message box.



Finding the main() Function in Ghidra

Ghidra doesn't land directly on `main()` when you open a binary. Windows executables start from an `entry` point set by the linker, which calls into CRT (C runtime) startup code before reaching `main()`.

Answer the questions below

What is the only API call found in the User32.dll under the Imports section?

MessageBoxA (click on User32.dll)

How many times is the "Hello World" string found with the Search for Strings utility?

1 (search > for strings)

What is the virtual address of the CALL function that displays "Hello World" in a messagebox?

004073d7 (right click on messagebox > show reference to)

Task 5 - Identifying C Code Constructs in Assembly

There are different approaches to begin analyzing the disassembled code:

- Locate the main function from the **Symbol Tree** section.
- Check the **.text** code from the **Program Trees** section to see the code section and find the entry point.
- Search for interesting **strings** and locate the code from where those strings are referenced.

Note: Different compilers add their own code for various checks while compiling. Therefore expect some garbage assembly code that does not make sense.

Answer the questions below

What value gets printed by the while loop in the while-loop.exe program?
ITs_Fun_to_Learn_at_THM

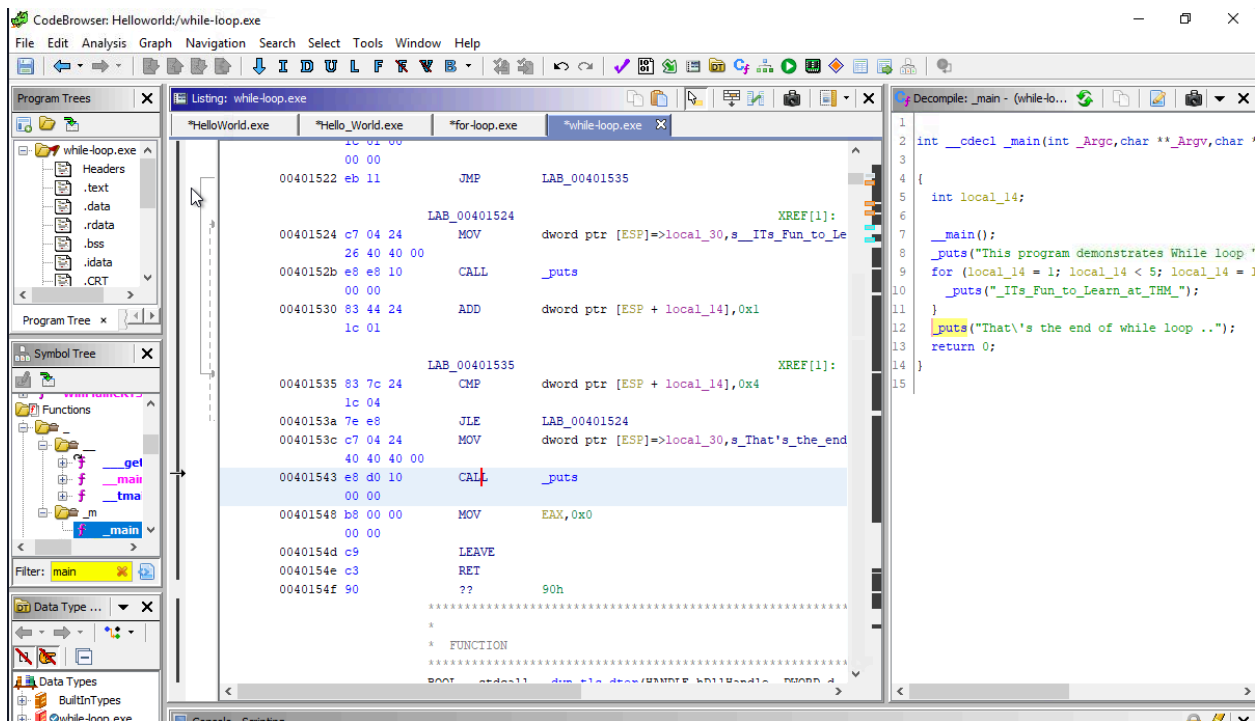
Go to **main** in the Symbol Tree

How many times, the while loop will run until the condition is met? 4

```
for (local_14 = 1; local_14 < 5; local_14 = local_14 + 1) {
```

Examine the while-loop.exe in Ghidra. What is the virtual address of the instruction, that CALLS to print out the sentence **"That's the end of while loop .."**?

00401543



In the if-else.exe program, examine the strings and complete the sentence

This program demonstrates if-else statement

What is the virtual address of the CALL to the main function in the if-else.exe program?

00401509 go to call function

Task 6 - An Overview of Windows API Calls

The Windows API is a collection of functions and services the Windows Operating System provides to enable developers to create Windows applications. These functions include creating windows, menus, buttons, and other user-interface elements and performing tasks such as file input/output and network communication.

Create Process API: The `CreateProcessA` Windows API function launches a new process by pushing parameters onto the stack in reverse order and calling the function, returning handles to the process and thread—analyzing this call helps reveal a malware's underlying intent.

Answer the questions below

When a process is created in suspended state, which hexadecimal value is assigned to the `dwCreationFlags` parameter?

0x00000004

Task 7 - Common APIs used by Malware

Malware authors heavily rely on Windows APIs to accomplish their goals. It's important to know the Windows APIs used in different malware variants. It's an important step in advanced static analysis to examine the `import` functions, which can reveal much about the malware.

Keylogger: Malware can use several Windows APIs for keylogging. Using these APIs, malware can intercept and record keystrokes, thereby capturing sensitive information such as passwords and credit card numbers.

Downloader: A downloader is a type of malware that downloads other malware onto a victim's system. Downloaders can be disguised as legitimate software or files and spread through malicious email attachments, software downloads, or by exploiting software vulnerabilities. Downloaders can use various Windows APIs to perform their malicious actions.

C2 Communication: Command and Control (C2) communication is a method malware uses to communicate with a remote server or attacker. This communication can be used to receive commands from the attacker, send stolen data to the attacker, or download additional malware onto the victim's system.

Data Exfiltration: Data exfiltration is the unauthorized transfer of data from an organization to an external destination. Malware can use various Windows APIs to perform data exfiltration.

Dropper: A dropper is a malware designed to install other malware onto a victim's system. Droppers can be disguised as legitimate software or files and spread via malicious email attachments, software downloads, or by exploiting software vulnerabilities.

API Hooking: API hooking is a technique malware uses to intercept Windows API calls and modify their behavior. This allows the malware to avoid detection by security software and perform malicious actions such as stealing data or modifying system settings.

Anti-debugging and VM detection: Anti-debugging and VM detection are techniques malware uses to evade detection and analysis by security researchers.

Task 8 - Process Hollowing: Overview

Process hollowing is a technique malware uses to inject malicious code into a legitimate process running on a victim's computer. The malware creates a suspended process and replaces its memory space with its own code. The malware then resumes the process, causing it to execute the injected code. This technique allows the malware to bypass security measures, as the malicious code is executed within the context of a legitimate process.

Process hollowing involves several steps:

- Create a new process using the `CreateProcessA()` API. This process will be legitimate and hollowed out.
- `NtSuspendProcess()` is then used to suspend the new process.
- Allocate memory in the suspended process using the `VirtualAllocEx()` API. This memory will be used to hold the malicious code.
- Write the malicious code to the allocated memory using the `WriteProcessMemory()` API.
- Modify the entry point of the process to point to the address of the malicious code using the `SetThreadContext()` and `GetThreadContext()` APIs.

- Resume the suspended process using the `NtResumeProcess()` API. This will cause the process to execute the malicious code.
- Clean up the process and any resources used during the process.

Answer the questions below

Which API is used to write malicious code to the allocated memory during process hollowing?

`WriteProcessMemory()`

Task 9 - Analyzing Process Hollowing

An important point to note is that almost all malware is packed with known or custom packers and employs various Anti-debugging/VM-detection techniques to hinder analysis. Our objective of advanced static analysis would be to:

- Examine the API calls to find a pattern or suspicious call.
- Look at the suspicious strings.
- Find interesting or malicious functions.
- Examine the disassembled/decompiled code to find as much information as possible.

Load the Sample: Load the program; it will show the summary

Analyze: Ghidra does not automatically launch at program start. It's up to us to pick which function we want to analyze first. We will start looking at the Windows APIs used to accomplish process hollowing.

Note: It's important to mention that starting to search for the `CreateProcessA` function right away is not how an analyst would start analyzing an unknown binary.

CreateProcess: Searching references for `CreateProcessA` reveals parameters pushed onto the stack in reverse order, where a creation flag value of `0x4` confirms the victim process is being spawned in a suspended state for process hollowing.

Graph View: Clicking the Display Function Graph in the toolbar displays the disassembled code.

- Fails to create a victim process in the suspended state; it will move to block 1. The **red arrow** represents the failure to meet the condition mentioned above.
- Successfully creates the victim process; it will move to block 2. The **green arrow** represents the success of the jump condition.

Open Suspicious File: Uses **CreateFileA** to create or open files, located via references in the Symbol Tree.

Hollow the Process: Uses **NtUnmapViewOfSection** or **ZwUnmapViewOfSection** to unmap and clear memory from the target process.

Allocate Memory: Uses **VirtualAllocEx** to allocate new memory within the hollowed process.

Write Down the Memory: Uses **WriteProcessMemory** to copy the malicious code sections into the allocated target process memory.

Resume thread: Uses **SetThreadContext** and **ResumeThread** to set the execution point and launch the injected payload.

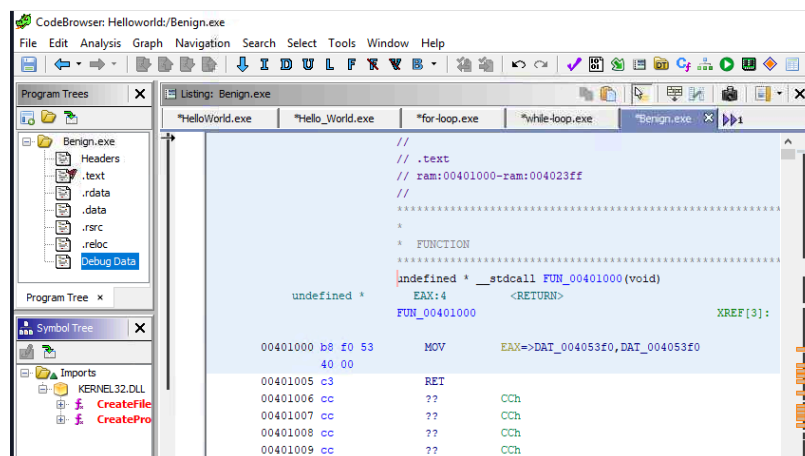
Answer the questions below

What is the MD5 hash of the benign.exe sample?

e60a461b80467a4b1187ae2081f8ca24

How many API calls are returned if we search for the term 'Create' in the Symbol Tree section?

2



What is the first virtual address where the CreateProcessA function is called?
0040108f

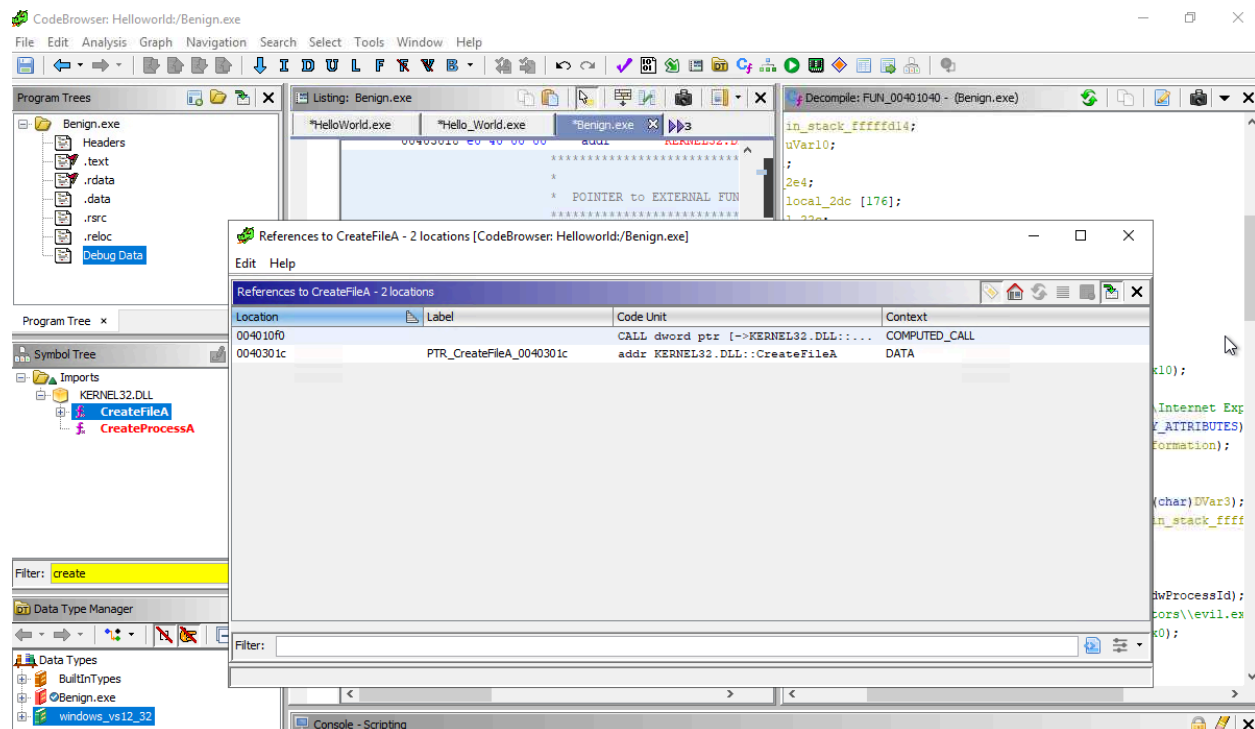
show reference to

Which process is being created in suspended state by using the CreateProcessA API call?

iexplore.exe

What is the first virtual address where the CreateFileA function is called?
004010f0

show reference to check decompile part



What is the suspicious process being injected into the victim process?

evil.exe

Based on the Function Graph, what is the virtual address of the code block that will be executed if the program doesn't find the suspicious process? 00401101

```
...10d9 ADD ESP,0xc
...10dc PUSH 0x0
...10de PUSH 0x0
...10e0 PUSH 0x3
...10e2 PUSH 0x0
...10e4 PUSH 0x1
...10e6 PUSH 0x80000000
...10eb PUSH s_C:\Users\THM-Attacker\De...
...10f0 CALL dword ptr [->KERNEL32.DLL:...
...10f6 MOV dword ptr [EBP + local_2e8...
...10fc CMP EAX,-0x1
...10ff JNZ LAB_00401134
```



```
00401101
...1101 CALL dword ptr [->KERNEL32.DLL:...
...1107 PUSH EAX
...1108 PUSH s_[-]_Unable_to_open_repla...
...110d CALL FUN_00401010
...1112 ADD ESP,0x8
...1115 PUSH 0x1
...1117 PUSH dword ptr [EBX]
...1119 CALL dword ptr [->KERNEL32.DLL:...
...111f POP ESI
...1120 MOV EAX,0x1
...1125 POP EBX
...1126 MOV ECX,dword ptr [EBP + local...
...1129 XOR ECX,EBP
...112b CALL FUN_0040148a
...1130 MOV ESP,EBP
...1132 POP EBP
...1133 RET
```

Which API call is found in the import functions used to unmap the process's memory?

NtUnmapViewOfSection

How many calls to the WriteProcessMemory function are found in the code?

2

What is the full path of the suspicious process shown in the strings?

C:\\Users\\THM-Attacker\\Desktop\\Injectors\\evil.exe

